

第 5 周 实验指导书——商城 Vue 管理后台

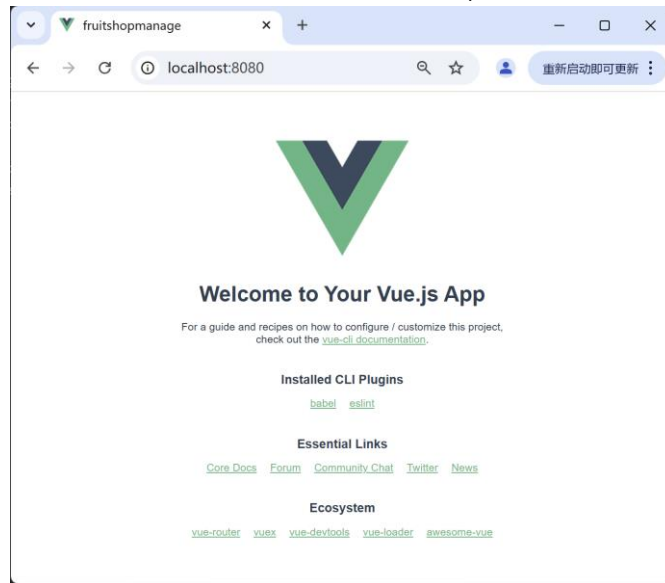
1. Vue 项目搭建

(1) 创建项目

步骤 1: 使用脚手架创建项目，使用 Manually select features，勾选 Router

```
vue create fruitshopmanage
```

步骤 2: 使用 vscode 打开项目，在终端中运行项目 npm run serve



步骤 3: 清理项目多余部分:

- 删除 views 下的 AboutView.vue 和 HomeView.vue
- 清理 router/index.js 下的内容

```
import { createRouter, createWebHashHistory } from 'vue-router'

const routes = [

]

const router = createRouter({
  history: createWebHashHistory(),
  routes
})

export default router
```

- 清理 App.vue 中的内容

```
<template>
  <div>
    <router-view/>
  </div>
</template>
```

```
<style>
#app {
  text-align: center;
  color: #2c3e50;
}
</style>
```

步骤 4: 创建组件 src/views 目录下创建 Login.vue

```
<template>
  <div>
    login
  </div>
</template>

<script setup>

</script>

<style scoped>

</style>
```

步骤 5: 创建组件 src/views 目录下创建 Main.vue

```
<template>
  <div>
    Main
  </div>
</template>

<script setup>

</script>

<style scoped>

</style>
```

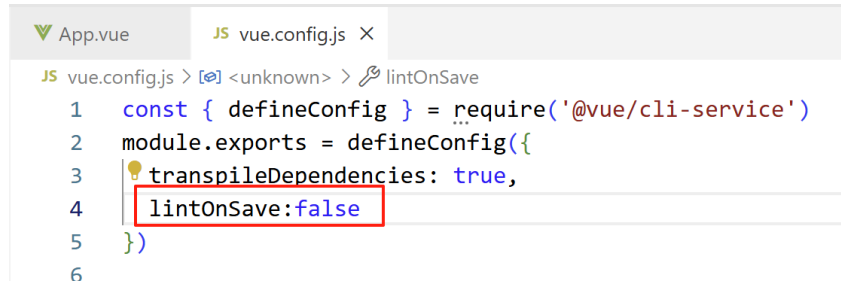
(2) 修改路由

步骤 1: 修改 src/router 下 index.js

```
import { createRouter,createWebHistory } from "vue-router";
import Login from "../views/Login.vue"
import Main from "../views/Main.vue"
const routes=[
  {path:"/login",name:"Login",component:Login},
  {path:"/",name:"Main",component:Main},
];
```

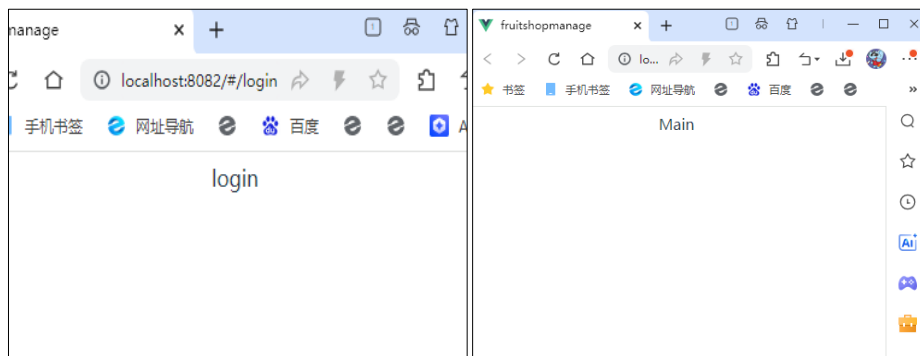
```
const router=createRouter({
  history:createWebHistory(),
  routes,
})
export default router
```

步骤 2: 如果出现 multi-word 错误, 在



```
JS vue.config.js > [?] <unknown> > lintOnSave
1  const { defineConfig } = require('@vue/cli-service')
2  module.exports = defineConfig({
3    transpileDependencies: true,
4    lintOnSave: false
5  })
6
```

步骤 3: 测试路由



步骤 6: 创建业务组件

在 views 下创建 index、product、user 文件夹, 在文件中创建如下组件。

- index/Index.vue
- product/ Category.vue
- product/ ProductList.vue
- user/UserList.vue

组件创建完后, 接下来创建路由, 修改 router.js 文件

```
import { createRouter,createWebHistory } from "vue-router";
import Login from "../views/Login.vue"
import Main from "../views/Main.vue"
import Index from "../views/index/Index.vue"
import Category from "../views/product/Category.vue"
import ProductList from "../views/product/ProductList.vue"
import UserList from "../views/user/UserList.vue"

const routes=[
  {path:"/login",name:"Login",component:Login},
  { path: "/",component: Main,
    children: [
      {path: 'index', component: Index},
      {path: 'category', component: Category},
```

```

        {path: 'product-list', component: ProductList},
      ],
    },
  ];

  const router=createRouter({
    history:createWebHistory(),
    routes,
  })
  export default router

```

(3) 使用 Element-Plus 制作组件

步骤 1: Element-Plus 的官网地址: <https://element-plus.sctxhy.com/zh-CN/>

步骤 2: 安装: `npm install element-plus --save`

步骤 3: 挂载到 Vue 实例, main.js 中修改

```

import { createApp } from 'vue'
import ElementPlus from 'element-plus' //引入 Element-plus
import 'element-plus/dist/index.css' //引入 Element-plus
import App from './App.vue'
import router from './router'

createApp(App).use(router).use(ElementPlus).mount('#app')

```

2. 登录页面及其功能的实现

(1) 安装并设置 Axios

步骤 1: Axios 官网地址: <http://www.axios-js.com/zh-cn/docs/>

步骤 2: Axios 安装: `npm install axios`

步骤 3: vue-axios 安装: `npm install --save axios vue-axios`

步骤 4: 在 main.js 中将 Axios 挂载到 Vue 实例中

```

import { createApp } from 'vue'
import ElementPlus from 'element-plus' //引入 Element-plus
import 'element-plus/dist/index.css' //引入 Element-plus
import App from './App.vue'
import router from './router'

//引入 axios
import axios from "axios"
import VueAxios from "vue-axios"

const app=createApp(App)
app.use(VueAxios,axios)
app.provide("axios",app.config.globalProperties.axios)

app.use(router).use(ElementPlus).mount('#app')

```

(2) 实现登录和退出功能

步骤 1: 在 Login.vue 中输入以下代码

```

<template>
  <div class="container">
    <div class="login-box">
      <div class="left-box">
        
      </div>
      <div class="right-box">
        <div class="logo">
          
        </div>
        <p>百果园管理后台</p>
        <div class="input-box">
          
          <el-input v-model="account" placeholder="请输入账号" class="input-
inner"/>
        </div>
        <div class="input-box">
          
          <el-input v-model="password" placeholder="请输入密码" class="input-
inner"/>
        </div>
        <el-button type="primary" class="login-btn" @click="login">登录</el-
button>
        <el-button @click="logout" class="login-btn">退出</el-button>
      </div>
    </div>
  </div>
</template>

<script setup>

</script>

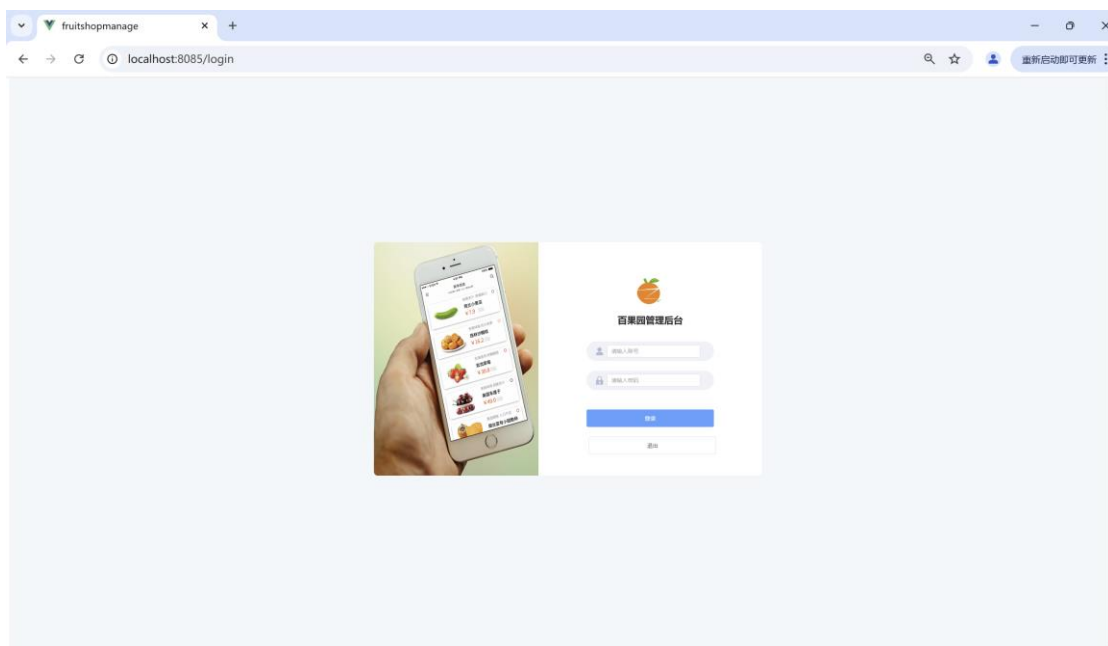
<style scoped>
.container{
  width: 100vw;
  height: 100vh;
  background-color: #f3f7fa;
  display: flex;
  justify-content: center;
  align-items: center;
}
.login-box{
  width: 1000px;

```

```
height: 600px;
background-color: #fff;
border-radius: 10px;
overflow: hidden;
display: flex;
}
.left-box{
width: 424px;
background-color: #333;
}
.left-box img{
width: 100%;
height: 100%;
}
.right-box{
flex: 1;
display: flex;
flex-direction: column;
align-items: center;
}
.logo{
width: 70px;
height: 70px;
margin-top: 90px;
border-radius: 50%;
overflow: hidden;
}
.logo img{
width: 100%;
height: 100%;
}
.right-box p{
font-size: 24px;
font-family: Microsoft YaHei-Bold, Microsoft YaHei;
font-weight: bold;
color: #272727;
margin-bottom: 40px;
}
.input-box{
width: 328px;
height: 50px;
background-color: #f1f2f7;
border-radius: 25px;
margin-bottom: 25px;
```

```
display: flex;
align-items: center;
}
.prefix-icon{
width: 23px;
height: 23px;
margin: 0 8px 0 22px;
}
.input-inner{
width: 240px;
}
.login-btn{
width: 328px;
height: 45px;
margin-top: 25px;
box-sizing: border-box;
}
</style>
```

步骤 2：测试外观



步骤 3：解决跨域问题

- 在后端 API 的 Express 框架程序中执行：npm install cors
- 导入中间件和配置中间件：在 app.js 中

// 设置跨域和相应数据格式，注意位置

```
const cors=require('cors');
```

```
app.use(cors());
```

```

1 var express=require('express')
2 var app=express()
3 //处理文件路径的小工具
4 var path=require('path')
5
6 //挂载中间件, 设置统一响应
7 var resextra=require('./modules/resExtra')
8 app.use(resextra)
9
10 // 设置跨域和相应数据格式
11 const cors=require('cors');
12 app.use(cors());
13
14 // 引入bodyParser解析json
15 var bodyParser=require('body-parser')
16 app.use(bodyParser.json())
17 app.use(bodyParser.urlencoded({extended:true}))
18

```

● 重新运行后端程序

步骤 4: 修改 Login.vue 的 script 代码

```

<script setup>
import axios from "axios";
import{ref,reactive} from 'vue'
import{ElMessage} from 'element-plus'
import{useRouter,useRoute} from 'vue-router'

const router=useRouter()
const route=useRoute()
const account = ref()
const password = ref()
const login=async()=>{
  // 数据校验
  if (account.value == "" || password.value == "") {
    ElMessage.error("请输入用户名和密码");
    return;
  }
  let loginForm = {
    username: account.value,
    password: password.value,
  };
  try {
    const { data: res } = await axios.post(`http://127.0.0.1:8088/sysapi/login`,
loginForm);
    console.log(res);
    if (res.meta.status !== 200) {
      ElMessage.error("登录失败");
      return
    }
    ElMessage.success("登录成功");
    window.localStorage.setItem("token", res.data.token);
    router.push('/index')
  } catch (e) {

```

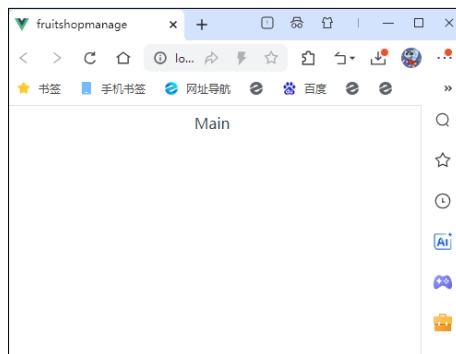


```

        console.log("登录失败"+e);
        ElMessage.error("网络错误或服务异常");
    }
}
const logout=async()=>{
    //退出
    window.sessionStorage.clear();
    router.push('/login')
}
</script>

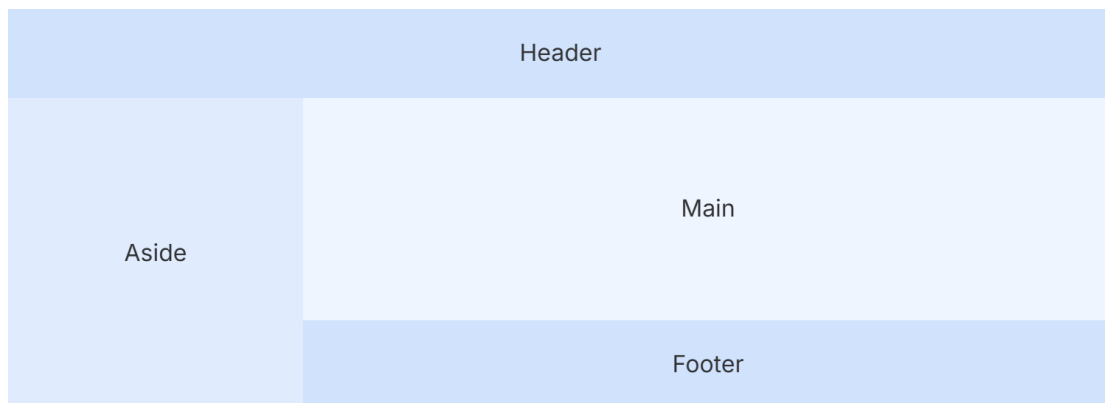
```

步骤 5: 测试, 输入 admin 123456 后显示登录成功, 打开下图



3.修改 Main.vue 主页面

步骤 1: 删掉 Main.vue 原来的内容, 使用 elementplus 的 container 布局



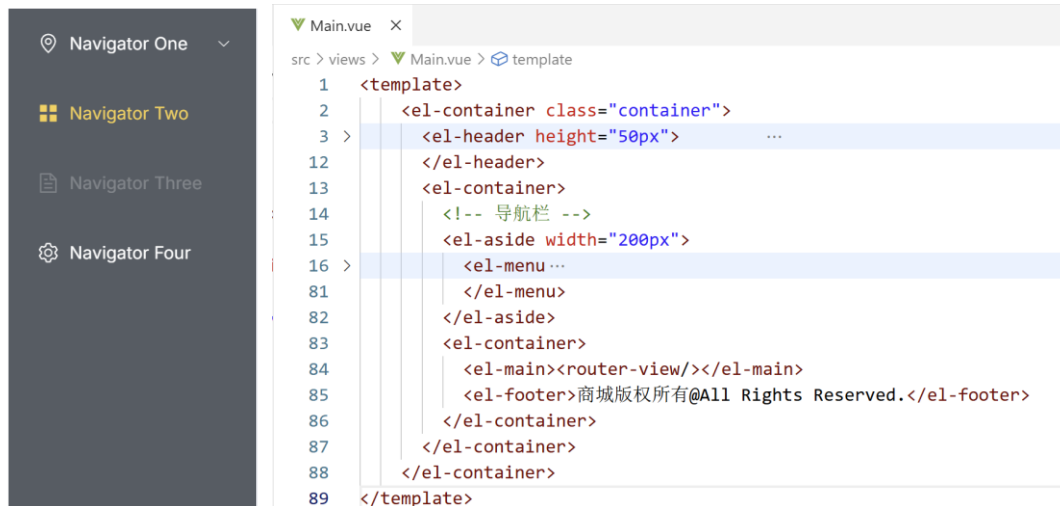
```

<template>
  <div class="common-layout">
    <el-container>
      <el-header>Header</el-header>
      <el-container>
        <el-aside width="200px">Aside</el-aside>
        <el-container>
          <el-main>Main</el-main>
          <el-footer>Footer</el-footer>
        </el-container>
      </el-container>
    </div>
  </template>

```

```
    </el-container>
  </div>
</template>
```

步骤 2: 左侧边栏, 使用 elementplus 的 Menu 菜单, 修改左侧导航后代码如下:



步骤 3: 设置显示图标

安装: npm install @element-plus/icons-vue

注册所有图标, 在 main.js 中输入下面代码:

```
// 如果您正在使用 CDN 引入, 请删除下面一行。
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}
```

步骤 4: Main.vue 完整的代码如下:

```
<template>
  <el-container class="container">
    <el-header height="50px">
      <div class="sys-name">百果园微信商城-管理后台</div>
      <div class="right-box">
        <a href="#"><el-icon><ChatDotSquare /></el-icon></a>
        <a href="#"><el-icon><Setting /></el-icon></a>
        <el-button type="warning" circle><el-icon><User /></el-icon></el-
button>
        <span>user</span>
        <a href="#" @click="logout"><i class="el-icon-switch-
button"></i></a>
      </div>
    </el-header>
    <el-container>
      <!-- 导航栏 -->
      <el-aside width="200px">
        <el-menu
```

```

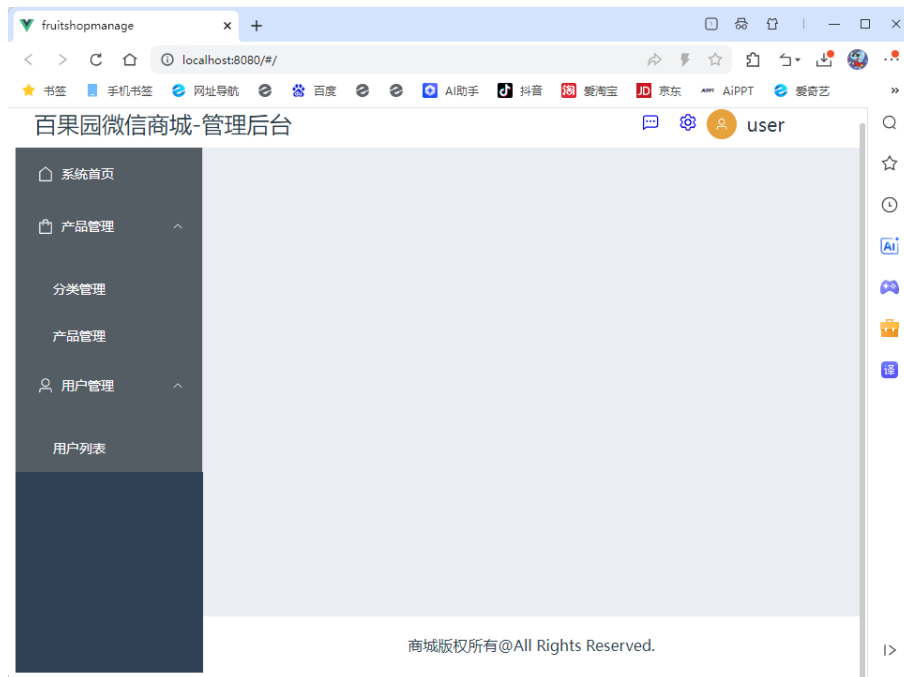
        active-text-color="#ffd04b"
        background-color="#545c64"
        class="el-menu-vertical-demo"
        text-color="#fff"
        @open="handleOpen"
        @close="handleClose"
      >
      <el-menu-item index="1">
        <el-icon><House /></el-icon>
        <span>系统首页</span>
      </el-menu-item>
      <el-sub-menu index="2">
        <template #title>
          <el-icon><Handbag /></el-icon>
          <span>产品管理</span>
        </template>
        <el-menu-item-group>
          <el-menu-item index="">分类管理</el-menu-item>
          <el-menu-item index="">产品管理</el-menu-item>
        </el-menu-item-group>
      </el-sub-menu>
      <el-sub-menu index="3">
        <template #title>
          <el-icon><User /></el-icon>
          <span>用户管理</span>
        </template>
        <el-menu-item-group>
          <el-menu-item index="">用户列表</el-menu-
item>
          </el-menu-item-group>
        </el-sub-menu>
      </el-menu>
    </el-aside>
    <el-container>
      <el-main><router-view/></el-main>
      <el-footer>商城版权所有@All Rights Reserved.</el-footer>
    </el-container>
  </el-container>
</template>

<style scoped>
.container{
  height: 100vh;

```

```
}  
.el-header{  
  background-color: #fff;  
  font-size: 20px;  
  display: flex;  
  justify-content: space-between;  
  padding-right: 50px;  
  line-height: 50px;  
}  
.sys-name{  
  font-size: 24px;  
}  
.right-box{  
  display: flex;  
  align-items: center;  
  justify-content: space-around;  
}  
.right-box i,  
.right-box span {  
  margin: 0 10px;  
}  
.el-aside {  
  background-color: #304156;  
  color: #fff;  
}  
.el-main {  
  background-color: #eaedf1;  
}  
.el-footer {  
  background-color: #fff;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
.el-menu-item.is-active {  
  background-color: rgb(67.74, 80) !important;  
}  
</style>
```

显示效果：



4.分类管理功能的实现

(1) 获取分类列表

步骤 1: 在 Main.vue 中设置路由跳转:

```

Main.vue x
src > views > Main.vue > template > el-container.container > el-container > el-aside > el-menu.el-menu-vertical-demo > el-sub-menu > el-menu-item-
1 <template>
2   <el-container class="container">
13   <el-container>
14     <el-aside width="200px">
15       <el-menu
20         text-color="#fff"
21         router
22         @open="handleOpen"
23         @close="handleClose"
24       >
25         <el-menu-item index="/index">
26           <el-icon><House /></el-icon>
27           <span>系统首页</span>
28         </el-menu-item>
29         <el-sub-menu index="2">
30           <template #title>
31             <el-icon><Handbag/></el-icon>
32             <span>产品管理</span>
33           </template>
34           <el-menu-item-group>
35             <el-menu-item index="/category">分类管理</el-menu-item>
36             <el-menu-item index="">产品管理</el-menu-item>
37           </el-menu-item-group>
38         </el-sub-menu>

```

步骤 2: 修改 Category.vue 文件, 添加 ElementPlus 中卡片

```
<template>
```

```

    <el-card class="box-card">
    </el-card>
  </template>

  <script>
  export default {
  }
  </script>

  <style scoped>
  .box-card {
    width: 100%;
  }
  </style>

```

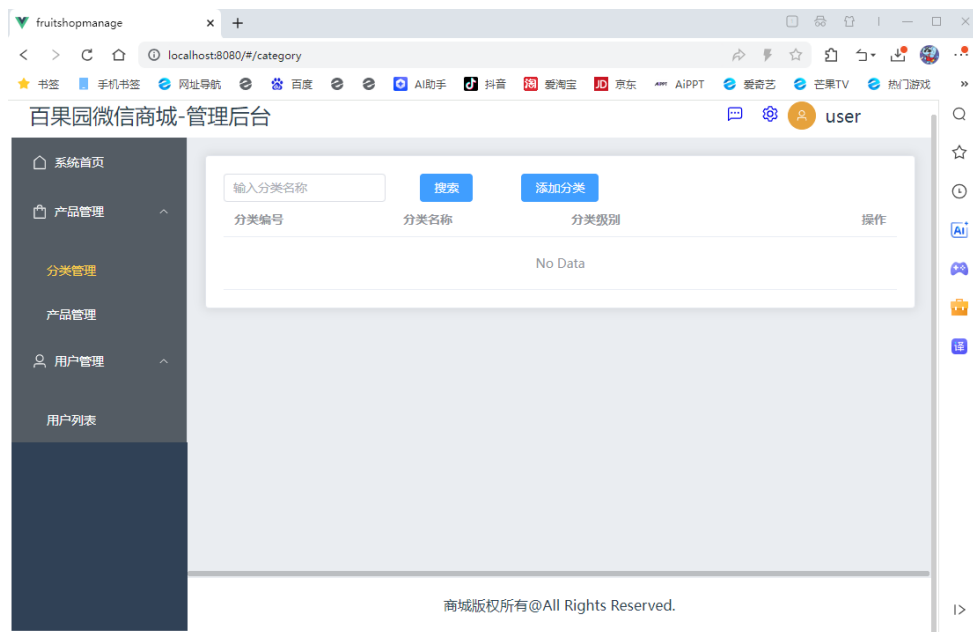
步骤 3: 使用 ElementPlus 中 Table 表中的自定义表头

```

<template>
  <el-card class="box-card">
    <!-- 搜索栏 -->
    <el-row :gutter="10">
      <el-col :span="6">
        <el-input placeholder="输入分类名称"></el-input>
      </el-col>
      <el-col :span="4">
        <el-button type="primary">搜索</el-button>
      </el-col>
      <el-col :span="4">
        <el-button type="primary" @click="showAddDialog">添加分类</el-
button>
      </el-col>
    </el-row>
    <el-table :data="filterTableData" style="width: 100%">
      <el-table-column label="Date" prop="date" />
      <el-table-column label="Name" prop="name" />
      <el-table-column align="right">
        <template #default="scope">
          <el-button size="small" @click="handleEdit(scope.$index, scope.row)">
编辑</el-button>
          <el-button size="small" type="danger"
@click="handleDelete(scope.$index, scope.row)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>
  </el-card>

```

</template>



步骤 4: 修改表格的 data 值为: catelist

修改表格的列为:

```
<el-table :data="catelist" style="width: 100%">
  <el-table-column label="分类编号" prop="cat_id" />
  <el-table-column label="分类名称" prop="cat_name" />
  <el-table-column label="分类级别" prop="cat_level" />
  <el-table-column label="操作" align="right">
    <template #default="scope">
      <el-button size="small" @click="handleEdit(scope.$index, scope.row)">
编辑</el-button>
      <el-button size="small" type="danger"
@click="handleDelete(scope.$index, scope.row)">删除</el-button>
    </template>
  </el-table-column>
</el-table>
```

步骤 5: 处理 catelist, 从后端 API 中获取数据

```
<script setup>
import axios from 'axios'
import {onMounted, ref} from 'vue'
const catelist=ref([])

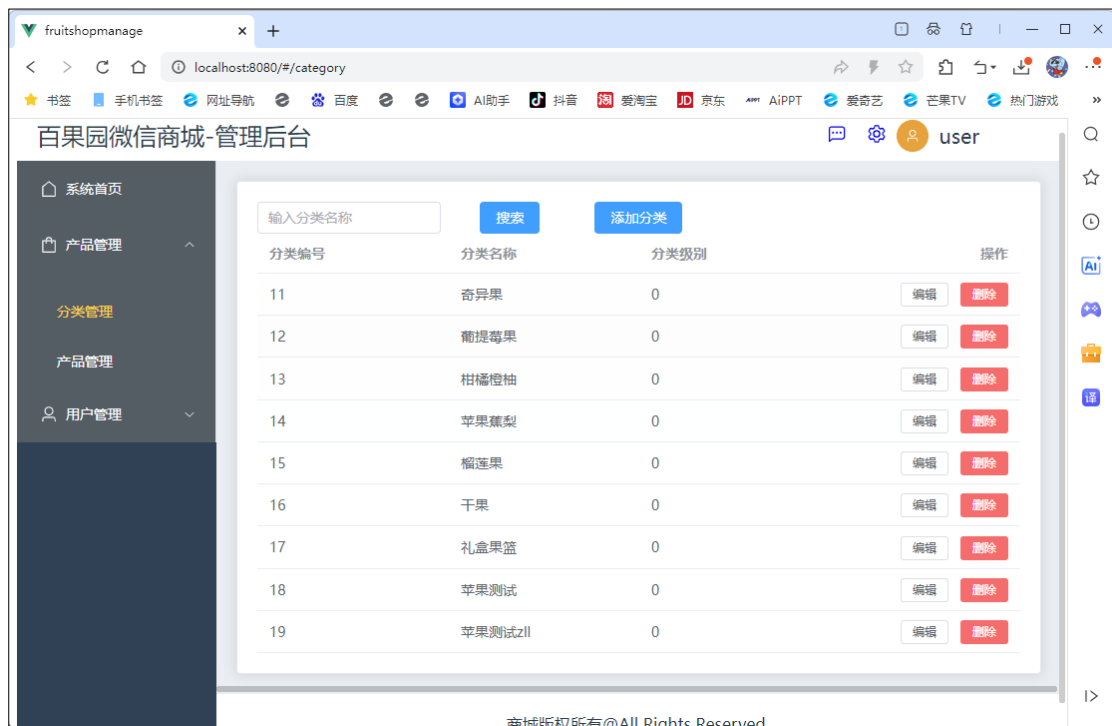
//获取类别列表
const getCateList=async()=>{
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.get('http://127.0.0.1:8088/sysapi/category', {
      headers: {
```

```

        'Authorization': `${token}` // 最常见的格式
    }
  });
  console.log(res);
  cateList.value=res.data
}catch(error){
  console.error("获取分类失败"+error)
}
}

//生命周期钩子函数
onMounted(()=>{
  getCateList()
})
</script>

```



(2) 添加分类

步骤 1: 使用 ElementPlus 中对话框中套表单，代码如下

```

<!-- 添加分类弹出层 begin -->
<el-dialog
  v-model="add_dialogVisible"
  title="添加分类"
  width="40%"
  :before-close="handleClose"
>
  <el-form
    ref="ruleFormRef"

```



```

      :model="addCateForm"
      status-icon
      :rules="rules"
      label-width="120px"
      class="demo-ruleForm"
    >
      <el-form-item label="分类名称: ">
        <el-input v-model="addCateForm.cat_name" autocomplete="off" />
      </el-form-item>
      <el-form-item label="父级分类: ">
        <el-select
          v-model="selectvalue"
          class="m-2"
          placeholder="请选择"
          style="width: 240px"
        >
          <el-option
            v-for="item in parentCateList"
            :key="item.cat_id"
            :label="item.cat_name"
            :value="item.cat_id"
          />
        </el-select>
      </el-form-item>
      <el-form-item>
        <el-button type="primary" @click="addCate">确定</el-button>
        <el-button @click="addCateDialogVisible = false">取消</el-button>
      </el-form-item>
    </el-form>
  </el-dialog>
  <!-- 添加分类弹出层 end -->

```

步骤 2: 添加分类的 Js 代码

```

<script setup>
import axios from 'axios'
import {onMounted, reactive, ref} from 'vue'
import {ElMessage} from 'element-plus'
const catelist=ref([])
const parentCateList=ref([])
const add_dialogVisible=ref(false)
const selectvalue = ref()
const addCateForm=reactive({
  cat_id:"",
  cat_name: "",// 将要添加的分类的名称

```

```

        cat_pid: 0, // 父级分类的 Id
        cat_level: 0, // 分类的等级, 默认要添加的是 1 级分类
    })

    // "添加分类" 按钮事件
    function showAddDialog() {
        add_dialogVisible.value = true;
    }

    // 点击按钮, 添加新的分类
    const addCate = async () => {
        const token = localStorage.getItem('token');
        try {
            const { data: res } = await axios.post(`http://127.0.0.1:8088/sysapi/category`, {
                cat_pid: selectvalue.value || 0, // 父级分类的 Id
                cat_name: addCateForm.cat_name, // 将要添加的分类的名称
                cat_level: addCateForm.cat_level, // 分类的等级, 默认要添加的是 1 级分类
            }, {
                headers: {
                    'Authorization': `${token}` // 最常见的格式
                }
            });
            if (res.meta.status !== 201) {
                ElMessage.error("添加分类失败!");
                return;
            }
            ElMessage.success("添加分类成功!");
            getCateList();
            add_dialogVisible.value = false;
        } catch (error) {
            console.error("获取分类失败" + error);
        }
    }

    // 获取类别列表
    const getCateList = async () => {
        const token = localStorage.getItem('token');
        try {
            const { data: res } = await axios.get(`http://127.0.0.1:8088/sysapi/category`, {
                headers: {
                    'Authorization': `${token}` // 最常见的格式
                }
            });
        }
    };

```

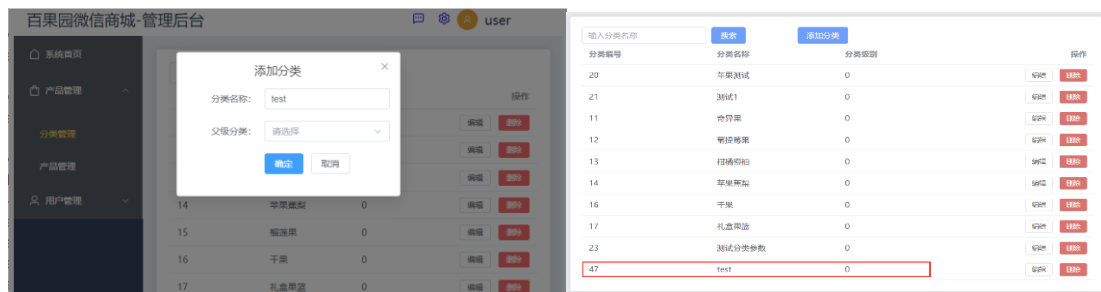
```

    console.log(res);
    catelist.value=res.data
    parentCateList.value=res.data
  }catch(error){
    console.error("获取分类失败"+error)
  }
}

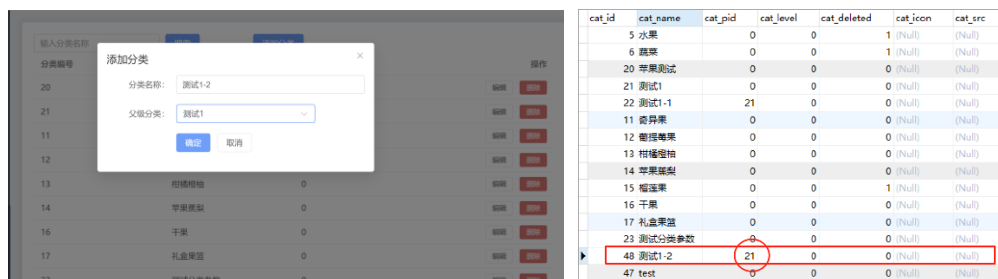
//生命周期钩子函数
onMounted(()=>{
  getCateList()
})
</script>

```

步骤 3：测试：添加一级分类



步骤 4：测试：添加二级分类【注意：二级分类在列表中看不到，可以更换 tree-table 组件可以查看二级分类，或者在数据库中可以看到二级分类】



(3) 删除分类

步骤 1：Html 的变化

```

<el-table :data="catelist" style="width: 100%">
  <el-table-column label="分类编号" prop="cat_id" />
  <el-table-column label="分类名称" prop="cat_name" />
  <el-table-column label="分类级别" prop="cat_level" />
  <el-table-column label="操作" align="right">
    <template #default="scope">
      <el-button size="small" @click="handleEdit(scope.$index, scope.row)">
        编辑</el-button>
      <el-button size="small" type="danger"
        @click="deleteCategory(scope.row.cat_id)">删除</el-button>
    </template>
  </el-table-column>

```

</el-table>

步骤 2: js 代码

```
//删除分类
const deleteCategory=async(cat_id)=>{
  const token = localStorage.getItem('token');
  if (cat_id == "") {
    ElMessage.error("为获取需要删的 ID");
    return;
  }
  try{
    const { data: res } = await
    axios.delete(`http://127.0.0.1:8088/sysapi/category/${cat_id}`, {
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    if (res.meta.status !== 400) {
      ElMessage.error("删除分类失败! ");
      return
    }
    ElMessage.success("删除分类成功! ");
    getCateList();
  }catch(error){
    console.error("删除分类失败"+error)
  }
}
```

步骤 3: 测试

cat_id	cat_name	cat_pid	cat_level	cat_deleted	cat_icon	cat_src
5	水果	0	0	0	1 (Null)	(Null)
6	蔬菜	0	0	0	1 (Null)	(Null)
20	苹果测试	0	0	0	0 (Null)	(Null)
21	测试1	0	0	0	0 (Null)	(Null)
22	测试1-1	21	0	0	0 (Null)	(Null)
11	奇异果	0	0	0	0 (Null)	(Null)
12	葡萄莓果	0	0	0	0 (Null)	(Null)
13	柑橘橙柚	0	0	0	0 (Null)	(Null)
14	苹果蕉梨	0	0	0	0 (Null)	(Null)
15	榴莲果	0	0	0	1 (Null)	(Null)
16	干果	0	0	0	0 (Null)	(Null)
17	礼盒果篮	0	0	0	0 (Null)	(Null)
23	测试分类参数	0	0	0	0 (Null)	(Null)
48	测试1-2	21	0	0	0 (Null)	(Null)
47	test	0	0	0	1 (Null)	(Null)

(4) 修改分类

步骤 1: 修改按钮

```
<el-button size="small" @click="editCategory(scope.row.cat_id)">编辑</el-button>
```

步骤 2: 编辑窗口

```
<!-- 编辑分类弹出层 begin -->
<el-dialog
  v-model="edit_dialogVisible"
  title="修改分类"
  width="40%" :before-close="handleClose">
  <el-form
    ref="ruleFormRef"
    :model="editCateForm"
    status-icon
    :rules="rules"
    label-width="120px"
    class="demo-ruleForm">
    <el-form-item label="分类名称: ">
      <el-input v-model="editCateForm.cat_name" autocomplete="off" />
    </el-form-item>
    <el-form-item>
      <el-button type="primary" @click="editCate(editCateForm.cat_id)">确定
</el-button>
      <el-button @click="edit_dialogVisible=false">取消</el-button>
    </el-form-item>
  </el-form>
</el-dialog>
<!-- 编辑分类弹出层 end -->
```

步骤 3: JS 代码

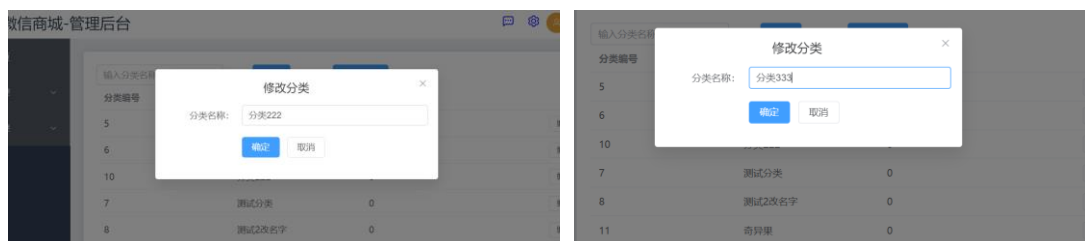
```
const edit_dialogVisible = ref(false);
const editCateForm = reactive({
  cat_id: "",
  cat_name: "",
});
//编辑分类，渲染原始分类名称
const editCategory = async (cat_id) => {
  edit_dialogVisible.value = true;
  const token = localStorage.getItem("token");
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/category/${cat_id}`, {
        headers: {
          Authorization: `${token}`, // 最常见的格式
        },
      }
    );
  }
};
```

```

    if (res.meta.status !== 200) {
      ElMessage.error("获取分类失败! ");
      return;
    }
    editCateForm.cat_name=res.data.cat_name
    editCateForm.cat_id=res.data.cat_id
  } catch (error) {
    console.error("获取分类失败" + error);
  }
}
//点击按钮，编辑分类
const editCate = async (cat_id) => {
  const token = localStorage.getItem("token");
  try {
    const { data: res } = await axios.put(
      `http://127.0.0.1:8088/sysapi/category/${cat_id}`,
      {
        cat_name: editCateForm.cat_name,
      },
      {
        headers: {
          Authorization: `${token}`, // 最常见的格式
        },
      }
    );
    if (res.meta.status !== 200) {
      ElMessage.error("更新分类失败! ");
      return;
    }
    ElMessage.success("更新分类成功! ");
    getCateList();
    edit_dialogVisible.value = false;
  } catch (error) {
    console.error("更新分类失败" + error);
  }
};

```

步骤 4：测试



输入分类名称	搜索	添加分类		
分类编号	分类名称	分类级别	操作	
5	水果	0	编辑	删除
6	蔬菜	0	编辑	删除
10	分类333	0	编辑	删除
7	测试分类	0	编辑	删除
8	测试2改名字	0	编辑	删除

5.商品管理功能的实现

(1) 获取商品列表

步骤 1: 修改 ProductList.vue, 先使用 ElementPlus 布局整个页面

```

<template>
  <el-card class="box-card">
    <template #header>
      <el-row :gutter="10">
        <el-col :span="4">
          <!-- 请选择商品分类 -->
          <el-select v-model="value" class="m-2" placeholder="请选择商品分类">
            <el-option
              v-for="item in options"
              :key="item.value"
              :label="item.label"
              :value="item.value"/>
          </el-select>
        </el-col>
        <el-col :span="6">
          <!-- 输入商品名称 -->
          <el-input v-model="input" placeholder="输入商品名称" />
        </el-col>
        <el-col :span="4">
          <el-button type="primary">搜索</el-button>
        </el-col>
        <el-col :span="4">
          <el-button type="primary">添加商品</el-button>
        </el-col>
      </el-row>
    </template>
    <el-table :data="filterTableData" style="width: 100%">
      <el-table-column label="Date" prop="date" />
      <el-table-column label="Name" prop="name" />
      <el-table-column align="right">
        <template #default="scope">
          <el-button size="small" @click="handleEdit(scope.$index, scope.row)">
            >Edit</el-button>
        </template>
      </el-table-column>
    </el-table>
  </el-card>
</template>

```

```

    >
    <el-button
      size="small"
      type="danger"
      @click="handleDelete(scope.$index, scope.row)"
    >Delete</el-button>
  >
</template>
</el-table-column>
</el-table>
</el-card>
</template>

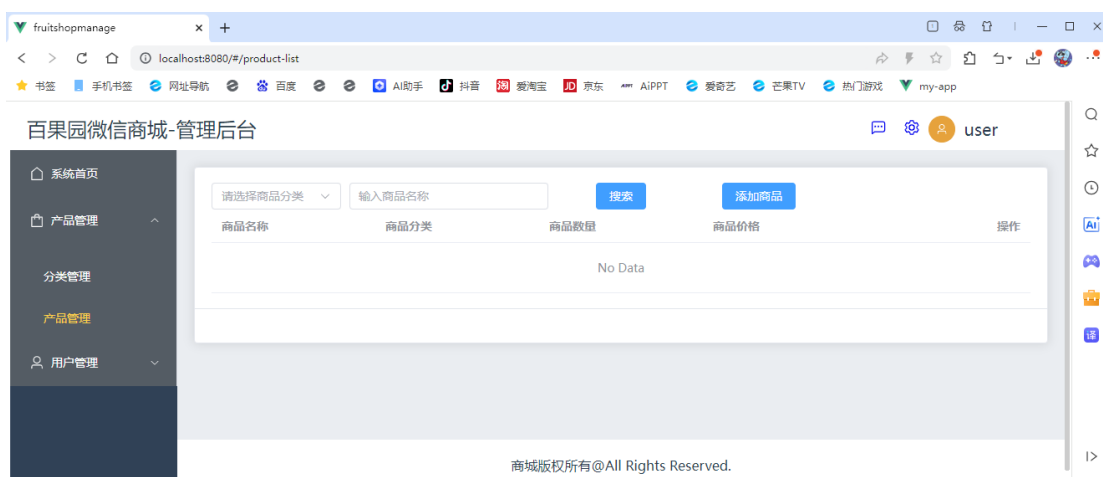
```

步骤 2: 修改 Main.vue 中的产品管理的 index 值, 使其能跳转到 ProductList 页面

```

<el-menu-item-group>
  <el-menu-item index="">品牌管理</el-menu-item>
  <el-menu-item index="/category">分类管理</el-menu-item>
  <el-menu-item index="">分类参数</el-menu-item>
  <el-menu-item index="/product-list">产品管理</el-menu-item>
</el-menu-item-group>

```



步骤 3: 实现表格数据的获取和渲染 HTML

```

<el-table :data="goods" style="width: 100%">
  <el-table-column label="商品名称" prop="goods_name" />
  <el-table-column label="商品分类" prop="cat_id"/>
  <el-table-column label="商品数量" prop="goods_number"/>
  <el-table-column label="商品价格" prop="goods_price"/>
  <el-table-column align="right" label="操作">
    <template #default="scope">
      <el-button size="small"
        @click="goUpdateProduct(scope.row.goods_id)">编辑</el-button>
      <el-button size="small" type="danger"
        @click="removeById(scope.row.goods_id)">删除</el-button>
    </template>
  </el-table-column>
</el-table>

```



```
</el-table-column>
</el-table>
```

步骤 3: 实现表格数据的获取和渲染 JS

```
<script setup>
import axios from 'axios'
import {onMounted, reactive, ref} from 'vue'
const goods=ref([])
const queryInfo=reactive({
  query:"",
  pagenum:1,
  pagesize:10,
})
//获取类别列表
const getGoodsList=async()=>{
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.get(`http://127.0.0.1:8088/sysapi/goods`, {
      params:queryInfo,
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    console.log(res);
    goods.value=res.data.goods
  }catch(error){
    console.error("获取商品失败"+error)
  }
}
//生命周期钩子函数
onMounted(()=>{
  getGoodsList()
})
</script>
```



步骤 4: 分页处理, 使用 elementPlus 中的 **Pagination** 分页, 放在</el-table>后面

```
<!-- 分页 -->
<el-pagination
  v-model:current-page="queryInfo.pagenum"
  v-model:page-size="queryInfo.pagesize"
  :page-sizes="[5, 10, 15, 20]"
  :small="small"
  :disabled="disabled"
  :background="background"
  layout="total, sizes, prev, pager, next, jumper"
  :total="total"
  @size-change="handleSizeChange"
  @current-change="handleCurrentChange"/>
```

步骤 5: 实现分页处理 JS

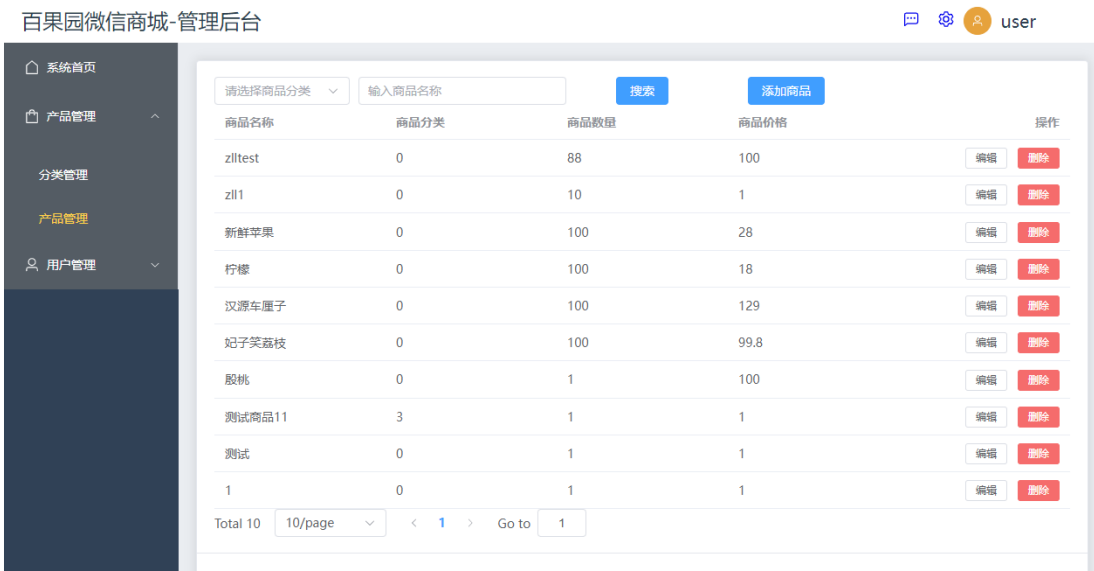
```
<script setup>
import axios from 'axios'
import {onMounted, reactive, ref} from 'vue'
const goods=ref([])
const queryInfo=reactive({
  query:"",
  pagenum:1,
  pagesize:10,
})
const total=ref(0)
//获取类别列表
const getGoodsList=async()=>{
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.get(`http://127.0.0.1:8088/sysapi/goods`, {
```

```

      params: queryInfo,
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    console.log(res);
    goods.value = res.data.goods
    total.value = res.data.total
  } catch (error) {
    console.error("获取商品失败" + error)
  }
}, handleSizeChange = (newSize) => {
  queryInfo.pagesize = newSize;
  getGoodsList();
}, handleCurrentChange = (newPage) => {
  queryInfo.pagenum = newPage;
  getGoodsList();
}

//生命周期钩子函数
onMounted(() => {
  getGoodsList()
})
</script>

```



(2) 搜索商品

步骤 1: 搜索商品 html

```

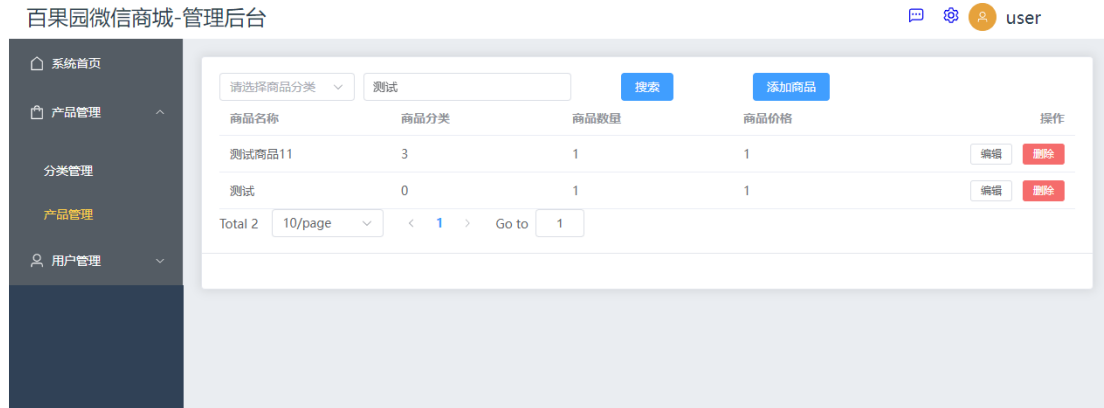
<el-col :span="6">
  <!-- 输入商品名称 -->
  <el-input v-model="queryInfo.query" placeholder="输入商品名称" />

```

```

</el-col>
<el-col :span="4">
    <el-button type="primary" @click="getGoodsList">搜索</el-button>
</el-col>

```



(3) 添加商品

步骤 1: 单击“添加商品”跳转到 AddProduct.vue

```

<el-col :span="4">
    <el-button type="primary" @click="goAddProduct">添加商品</el-
button>
</el-col>
.....
import {useRouter} from 'vue-router'
const router=useRouter()
.....
function goAddProduct(){
    router.push("/add-product");
}

```

步骤 2: 修改路由文件

```
import AccountList from "../views/sys/AccountList.vue"
import AddProduct from "../views/product/AddProduct.vue"
```

```
const routes = [
  {path: "/login", name: "Login", component: Login},
  {path: "/", component: Main,
    children: [
      {path: 'index', component: Index},
      {path: 'user-list', component: UserList},
      {path: 'category', component: Category},
      {path: 'brand', component: Brand},
      {path: 'category-param', component: CategoryParam},
      {path: 'product-list', component: ProductList},
      {path: 'order-list', component: OrderList},
      {path: 'user-report', component: UserReport},
      {path: 'product-report', component: ProductReport},
      {path: 'order-report', component: OrderReport},
      {path: 'account-list', component: AccountList},
      {path: 'add-product', component: AddProduct},
    ]
  }
]
```

步骤 2: 新建 AddProduct.vue 文件, 使用 ElementPlus 的 **Alert** 提示, 使用 ElementPlus 的 **Steps** 步骤条, 使用 ElementPlus 的 **Form** 表单中包含 **Tabs** 标签页。

```
<template>
  <!-- 卡片区域 -->
  <el-card>
    <!-- 提示区域 -->
    <el-alert title="添加商品信息" type="info" center show-icon />
    <!-- 步骤条区域 -->
    <el-steps :active="activeIndex - 0" finish-status="success">
      <el-step title="基本信息" />
      <el-step title="商品图片" />
      <el-step title="商品详情" />
      <el-step title="完成" />
    </el-steps>
    <!-- tab 栏区域 -->
    <el-form ref="formRef" :model="dynamicValidateForm" label-width="120px"
      class="demo-dynamic" style="margin-top:30px">
      <el-tabs :tab-position="tabPosition" style="height: 300px" class="demo-
        tabs">
        <el-tab-pane label="基本信息">

          </el-tab-pane>
          <el-tab-pane label="商品图片">

          </el-tab-pane>
          <el-tab-pane label="商品详情">
```

```

        </el-tab-pane>
      </el-tabs>
    </el-form>

  </el-card>
</template>

<script setup>
import { ref } from 'vue'

//Tabs 标签页的方向
const tabPosition = ref('left')

</script>

<style scoped>
.demo-tabs > .el-tabs__content {
  padding: 32px;
  color: #6b778c;
  font-size: 32px;
  font-weight: 600;
}
.el-tabs--right .el-tabs__content,
.el-tabs--left .el-tabs__content {
  height: 100%;
}
</style>

```

效果：



步骤 3：第一个选项卡“基本信息”，其中“商品类别”下拉列表框中渲染数据

```
<!-- tab 栏区域 -->
  <el-form ref="formRef" :model="addForm" label-width="120px" class="demo-
dynamic" style="margin-top:30px">
    <el-tabs :tab-position="tabPosition" style="height: 300px" class="demo-
tabs">
      <!-- 第一个选项卡 -->
      <el-tab-pane label="基本信息">
        <el-form-item label="商品名称" prop="goods_name">
          <el-input v-model="addForm.goods_name"></el-input>
        </el-form-item>
        <el-form-item label="商品价格" prop="goods_price">
          <el-input v-model="addForm.goods_price"
type="number"></el-input>
        </el-form-item>
        <el-form-item label="商品重量" prop="goods_weight">
          <el-input v-model="addForm.goods_weight"
type="number"></el-input>
        </el-form-item>
        <el-form-item label="商品数量" prop="goods_number">
          <el-input v-model="addForm.goods_number"
type="number"></el-input>
        </el-form-item>
        <el-form-item label="商品分类"
prop="goods_cat">
          <el-select
            v-model="addForm.goods_cat"
            class="m-2"
            placeholder="请选择"
            style="width: 240px"
          >
            <el-option
              v-for="item in catelist"
              :key="item.cat_id"
              :label="item.cat_name"
              :value="item.cat_id"
            />
          </el-select>
        </el-form-item>
      </el-tab-pane>
      <!-- 第二个选项卡 -->
      <el-tab-pane label="商品图片">

      </el-tab-pane>
      <!-- 第三个选项卡 -->
```

```
<el-tab-pane label="商品详情">

  </el-tab-pane>
</el-tabs>
</el-form>
```

步骤 4: 第一个选项卡“基本信息”，其中“商品类别”下拉列表框中渲染数据的 JS 代码

```
<script setup>
import axios from 'axios'
import {onMounted, reactive, ref} from 'vue'
import {ElMessage} from 'element-plus'

const addForm=reactive({
  goods_name: '',
  goods_price: 0,
  goods_weight: 0,
  goods_number: 0,
  // 商品所属的分类数组
  goods_cat: [],
  // 图片的数组
  pics: [],
  // 商品的详情描述
  goods_introduce: ''
})

//获取类别列表
const cateList=ref([])
const getCateList=async()=>{
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.get(`http://127.0.0.1:8088/sysapi/category`, {
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    console.log(res);
    cateList.value=res.data
    parentCateList.value=res.data
  }catch(error){
    console.error("获取分类失败"+error)
  }
}

//Tabs 标签页的方向
const tabPosition = ref('left')
```



```
//生命周期钩子函数
onMounted(()=>{
  getCateList()
})

</script>
```

步骤 5: 商品图片上传: upload 上传中的“图片列表缩略图”

```
<!-- 第二个选项卡 -->
    <el-tab-pane label="商品图片">
      <el-upload
        class="upload-demo"
        :http-request="uploadImage"
        :file-list="fileList"
        action="#"
        :on-preview="handlePreview"
        :on-remove="handleRemove"
        list-type="picture"
      >
        <el-button type="primary">点击上传</el-
button>
      </el-upload>
    </el-tab-pane>
```

步骤 6: 商品图片上传 JS, 能成功上传到服务器端

```
//图片上传
const uploadImage = async(option) => {
  const { file, onError } = option;
  const formData = new FormData();
  const token = localStorage.getItem('token');
  formData.append('file', file);
  try {
    const { data: res } = await axios.post('http://127.0.0.1:8088/sysapi/upload',
formData, {
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    console.log(res);
    addForm.pics.push({pic:res.data.tmp_path});
  } catch (err) {
    onError(err);
  }
};
```

步骤 7: 第三个选项卡“商品详情”——富文本编辑器 VueQuill

官网: <https://vueup.github.io/vue-quill/>

npm install @vueup/vue-quill@latest --save

全局注册 main.js

```
//富文本编辑器
import { QuillEditor } from '@vueup/vue-quill'
import '@vueup/vue-quill/dist/vue-quill.snow.css';
app.component('QuillEditor', QuillEditor)
```

步骤 8: 简单应用代码:

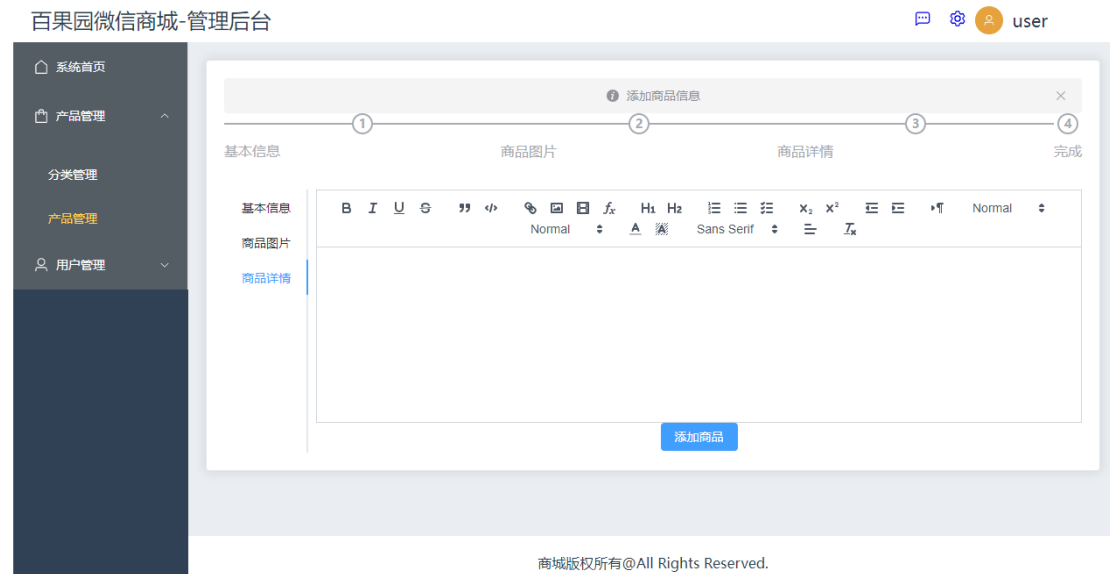
```
<el-tab-pane label="商品详情">
  <!-- 富文本编辑器组件 -->
  <QuillEditor theme="snow" />
  <!-- 添加商品的按钮 -->
  <el-button type="primary" class="btnAdd" @click="add">添加商
品</el-button>
</el-tab-pane>
```



步骤 9: 修改成一个具有复杂工具的富文本编辑器

```
<!-- 第三个选项卡 -->
<el-tab-pane label="商品详情">
  <!-- 富文本编辑器组件 -->
  <QuillEditor theme="snow" style="height:300px" v-
model="addForm.goods_introduce" :toolbar="[
    ['bold', 'italic', 'underline', 'strike'],
    ['blockquote', 'code-block'],
    ['link', 'image', 'video', 'formula'],
    [{ 'header': 1 }, { 'header': 2 }],
    [{ 'list': 'ordered' }, { 'list': 'bullet' }, { 'list': 'check' }],
    [{ 'script': 'sub' }, { 'script': 'super' }],
    [{ 'indent': '-1' }, { 'indent': '+1' }],
    [{ 'direction': 'rtl' }],
    [{ 'size': ['small', false, 'large', 'huge'] }],
    [{ 'header': [1, 2, 3, 4, 5, 6, false] }],
    [{ 'color': [] }, { 'background': [] }],
    [{ 'font': [] },
    [{ 'align': [] },
    ['clean']]"/>
  <!-- 添加商品的按钮 -->
  <el-button type="primary" class="btnAdd" @click="add">添加商
品</el-button>
```

</el-tab-pane>



步骤 10: 富文本编辑器获取值

```
<el-tab-pane label="商品详情">
  <!-- 富文本编辑器组件 -->
  <QuillEditor theme="snow" style="height:200px" v-
model:content="addForm.goods_introduce" contentType="html"
:toolbar="[
    ['bold', 'italic', 'underline', 'strike'],
    ['blockquote', 'code-block'],
    ['link', 'image', 'video', 'formula'],
    [{ 'header': 1 }, { 'header': 2 }],
    [{ 'list': 'ordered' }, { 'list': 'bullet' }, { 'list': 'check' }],
    [{ 'script': 'sub' }, { 'script': 'super' }],
    [{ 'indent': '-1' }, { 'indent': '+1' }],
    [{ 'direction': 'rtl' }],
    [{ 'size': ['small', false, 'large', 'huge'] }],
    [{ 'header': [1, 2, 3, 4, 5, 6, false] }],
    [{ 'color': [] }, { 'background': [] }],
    [{ 'font': [] }],
    [{ 'align': [] }],
    ['clean']]"/>
  <!-- 添加商品的按钮 -->
  <el-button type="primary" class="btnAdd" @click="add">添加商
品</el-button>
</el-tab-pane>
```

步骤 11: 编写“添加商品”按钮的单击事件，整个 JS 代码

```
<script setup>
import _ from "lodash";
import axios from 'axios'
```

```

import {onMounted, reactive, ref} from 'vue'
import {ElMessage} from 'element-plus'
import {useRouter,useRoute} from 'vue-router'
const router=useRouter()

const addForm=reactive({
  goods_name: '',
  goods_price: 0,
  goods_weight: 0,
  goods_number: 0,
  // 商品所属的分类数组
  goods_cat: [],
  // 图片的数组
  pics: [],
  // 商品的详情描述
  goods_introduce: ''
})

//获取类别列表
const catelist=ref([])
const getCateList=async()=>{
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.get(`http://127.0.0.1:8088/sysapi/category`, {
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    console.log(res);
    catelist.value=res.data
    parentCateList.value=res.data
  }catch(error){
    console.error("获取分类失败"+error)
  }
}

//图片上传
const uploadImage = async(option) => {
  const { file, onError } = option;
  const formData = new FormData();
  const token = localStorage.getItem('token');
  formData.append('file', file);
  try {
    const { data: res } = await axios.post('http://127.0.0.1:8088/sysapi/upload',

```

```

formData, {
  headers: {
    'Authorization': `${token}` // 最常见的格式
  }
});
console.log(res);
//addForm.pics=res.data.tmp_path
addForm.pics.push({pic:res.data.tmp_path});
} catch (err) {
  onError(err);
}
};

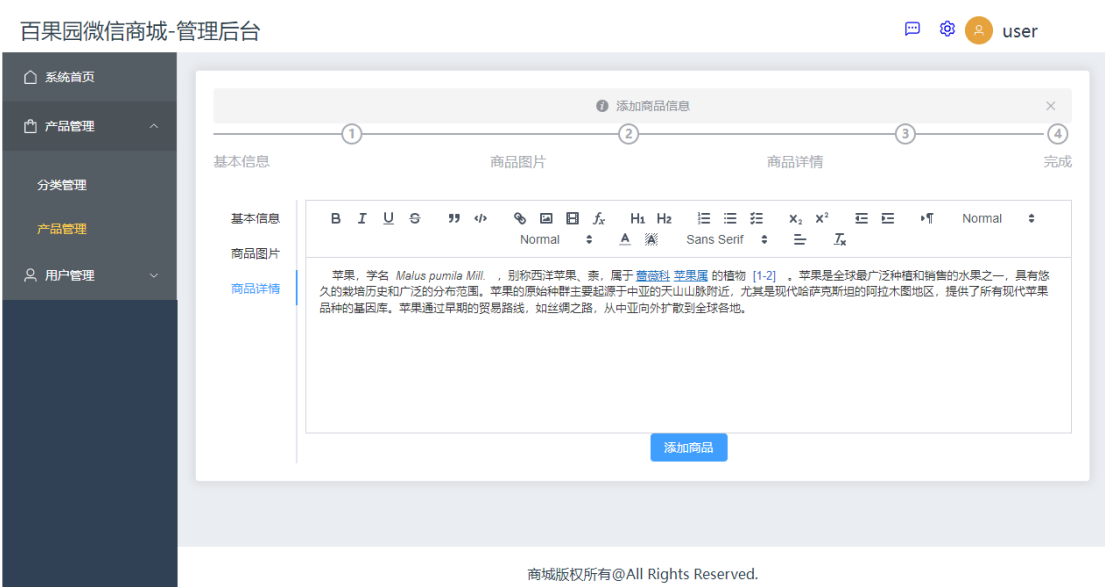
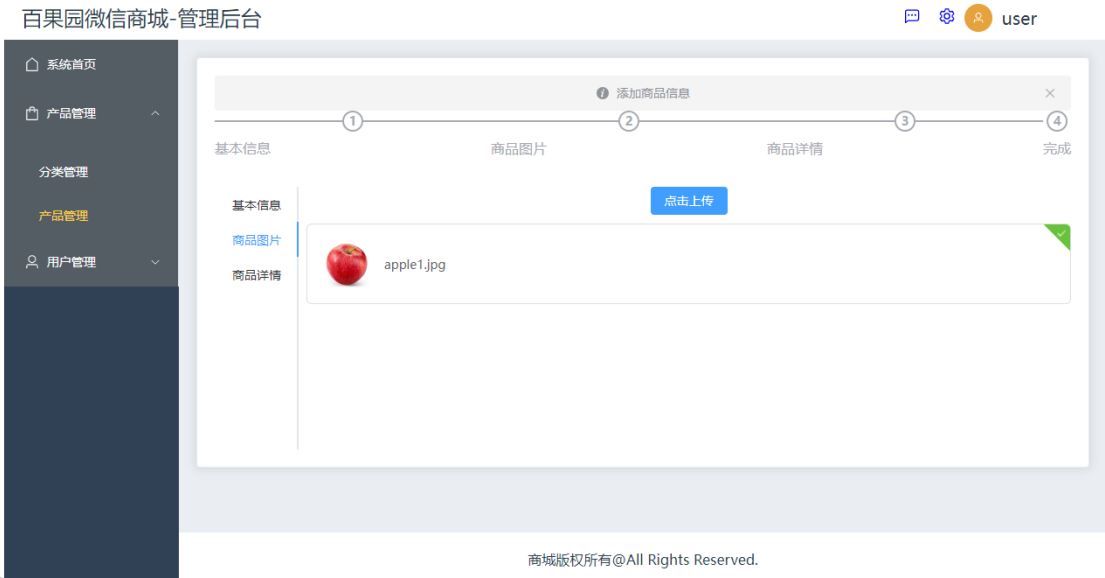
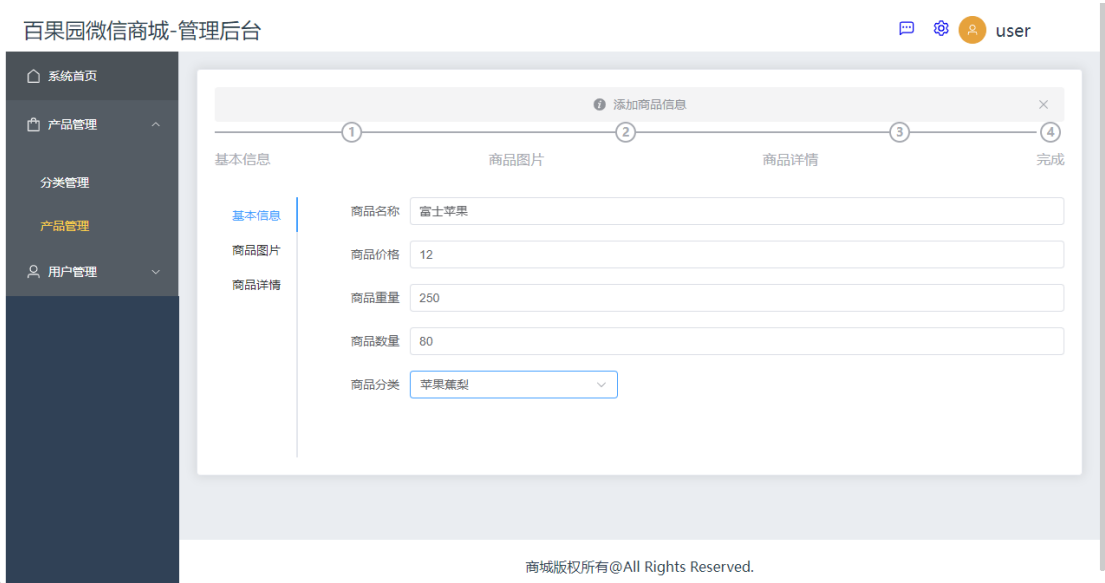
//添加商品
const add=async()=>>{
  addForm.goods_cat=addForm.goods_cat+"";
  const form = _cloneDeep(addForm);
  const token = localStorage.getItem('token');
  try{
    const { data: res } = await axios.post(`http://127.0.0.1:8088/sysapi/goods`, form,{
      headers: {
        'Authorization': `${token}` // 最常见的格式
      }
    });
    ElMessage.success("添加商品成功! ");
    router.push('/product-list')
  }catch(error){
    console.error("添加商品失败"+error)
  }
}

//Tabs 标签页的方向
const tabPosition = ref('left')

//生命周期钩子函数
onMounted(()=>{
  getCateList()
})
</script>

```

测试：



goods @fruit_shop (mysql) - 表

goods @fruit_shop (mys... x goods_pics @fruit_shop ... x

文件 编辑 查看 窗口 帮助

导入向导 导出向导 筛选向导 网格查看 表单查看 备注 十六进制 图像 升序排序 降序排序

goods_id	goods_name	goods_price	goods_number	goods_weight	cat_id	goods_introduce
9	测试商品11	1	1	0	3	test
10	殷桃	100	1	10	0	<p>樱桃很好吃哟</p><p><img src
11	1	1	1	1	0	<p>111
 删除</el-button>
```

步骤 2: js 部分

```
import { ElMessage } from 'element-plus'
.....
//删除商品
const removeById=async(goods_id)=>{
 const token = localStorage.getItem('token');
 if (goods_id == "") {
 ElMessage.error("未获取需要删的 ID");
 return;
```

```
}
try{
 const { data: res } = await
 axios.delete(`http://127.0.0.1:8088/sysapi/goods/${goods_id}`, {
 headers: {
 'Authorization': `${token}` // 最常见的格式
 }
 });
 if (res.meta.status !== 201) {
 ElMessage.error("删除商品失败! ");
 return
 }
 ElMessage.success("删除商品成功! ");
 getGoodsList();
} catch (error) {
 console.error("删除分类失败"+error)
}
}
```